

Adaptive Streaming Engine

HLS / DASH playback with bandwidth estimation and H.264 / HEVC / AV1 rendition switching

Project explained in simple terms: summary, components, workflow and architecture

The project line this document explains

Improved media playback performance, cutting time-to-first-frame 38% and rebuffering 42% in QoE telemetry, by building an HLS/DASH adaptive streaming engine with bandwidth estimation and H.264/HEVC/AV1 rendition switching.

What is inside this document

- **Section 1:** The big picture, an overall summary in plain language
- **Section 2:** Key terms explained simply (a mini dictionary)
- **Section 3:** Architecture diagrams, with every box and every arrow explained
- **Section 4:** Every component in detail, end to end
- **Section 5:** The complete workflow, end to end, step by step
- **Section 6:** How the 38% and 42% improvements were achieved and measured
- **Section 7:** How to explain the project in an interview

Note: thresholds, bitrates and timings quoted in this document are typical example values used to make the ideas concrete. Replace them with the exact values from your own implementation where they differ.

1. The Big Picture (Simple Summary)

When you watch a video on an app or website, the video is not downloaded as one big file. It is cut into many small pieces, each only a few seconds long, and each piece is stored in several quality levels (for example 240p, 480p, 720p, 1080p). The video player downloads the pieces one after another, just ahead of what you are watching.

The internet connection of a viewer is never steady. It goes up and down when you move between rooms, switch from Wi-Fi to mobile data, or when someone else in the house starts a download. If the player keeps asking for high-quality pieces while the connection is slow, the pieces arrive too late and the video freezes with a spinning wheel. If it always asks for low quality, the video looks blurry for no reason.

This project built the “brain” of the video player: an adaptive streaming engine. It reads the video's menu of available pieces (written in the two industry formats, **HLS** from Apple and **DASH** from the MPEG standard body), keeps measuring how fast the internet is (**bandwidth estimation**), watches how many seconds of video are already saved up (**the buffer**), and before every piece decides which quality and which compression format (**H.264, HEVC or AV1**) to download next. That decision is called **rendition switching**.

The engine also has special logic for the first second after you press Play, so the first picture appears quickly, and safety logic that drops quality early when the connection weakens, so the video does not freeze.

Every player reported what happened (how long startup took, how often it froze) through **QoE telemetry** (Quality of Experience measurements). Comparing the old player with the new engine showed:

Metric	What it means in plain words	Improvement
Time-to-first-frame (TTFF)	How long you wait after pressing Play before the first picture shows	38% faster (example: 2.1 s → 1.3 s)
Rebuffering	How much time the video spends frozen with a spinning wheel while you watch	42% less (example: 1.2% → 0.7% of watch time)

One-line analogy

The engine works like the automatic gearbox in a car. The road (the internet) keeps changing: uphill, downhill, traffic. The gearbox (the engine) keeps choosing the right gear (video quality) so the car (the video) never stalls and never wastes power.

1.1 The project line, decoded phrase by phrase

Phrase in the project line	What it means simply
Improved media playback performance	Made videos start faster and freeze less for real users.
Cutting time-to-first-frame 38%	The wait between pressing Play and seeing the first picture became 38% shorter.
Rebuffering 42%	The frozen, spinning-wheel time during playback dropped by 42%.
In QoE telemetry	These numbers come from real measurements sent back by players on viewers' devices, not from a lab guess.
HLS/DASH	The engine understands both popular streaming formats, so the same player works on all content and devices.
Adaptive streaming engine	The player logic that keeps adapting video quality to the viewer's network.
Bandwidth estimation	Continuously measuring how fast data is arriving, to predict what quality the connection can handle.
H.264/HEVC/AV1	Three compression formats (codecs). Newer ones (HEVC, AV1) give the same picture quality using fewer bits.
Rendition switching	Moving between quality levels (and codec versions) of the same video, piece by piece, without the viewer noticing.

2. Key Terms Explained Simply

These words appear throughout the document. Each is explained in one or two plain sentences.

Term	Simple meaning	Example
Streaming	Watching a video while it is still downloading, piece by piece.	Netflix, YouTube
Segment (chunk)	A small piece of the video, usually 2 to 6 seconds long.	seg_0042.m4s
Rendition	One complete version of the video at a particular quality and codec.	720p, H.264, 3 Mbps
Bitrate	How many bits per second a rendition needs. Higher bitrate means better picture but more data.	3,000 kbps = 3 Mbps
Bitrate ladder	The full list of renditions, from lowest to highest quality, like steps of a ladder.	240p → 1080p
Manifest (playlist)	A small text file that lists all renditions and where every segment is.	.m3u8 (HLS), .mpd (DASH)
HLS	HTTP Live Streaming. Apple's streaming format, used heavily on iPhone, Safari and smart TVs.	master.m3u8
DASH	Dynamic Adaptive Streaming over HTTP. The open MPEG standard, used widely on Android and web.	manifest.mpd
CMAF / fMP4	A shared container format, so one set of segment files can serve both HLS and DASH.	.m4s files
Codec	The method used to compress video. The device must have a matching decoder.	H.264, HEVC, AV1
H.264 (AVC)	The oldest and most widely supported codec. Works almost everywhere.	All phones and browsers
HEVC (H.265)	Newer codec, roughly 30–50% smaller files than H.264 at the same quality.	Apple devices, many TVs
AV1	Newest royalty-free codec, a further saving over HEVC. Needs newer hardware to decode efficiently.	Recent Android, Chrome, TVs
Bandwidth	How much data per second the viewer's connection is delivering right now.	8 Mbps on Wi-Fi
Buffer	Video that has been downloaded but not yet played. A savings account of seconds.	“12 seconds ahead”
ABR	Adaptive Bitrate. The logic that chooses which rendition to download next.	ABR Controller

Term	Simple meaning	Example
Rebuffering (stall)	When the buffer runs empty and the video freezes to wait for data.	Spinning wheel
TTF	Time-to-first-frame. Time from pressing Play to the first picture on screen.	1.3 seconds
QoE	Quality of Experience. How good watching felt to the viewer, measured with numbers.	Stalls, startup time
Telemetry	Measurements the player sends back to the company's servers.	“stall_start” event
CDN	Content Delivery Network. Many servers around the world holding copies of the video close to viewers.	Edge server in your city
MSE	Media Source Extensions. Browser feature that lets JavaScript feed video bytes to the built-in decoder.	SourceBuffer.appendBuffer()
Init segment	A tiny file with setup information the decoder needs before any video piece can play.	init.mp4
Keyframe (IDR)	A complete picture that does not depend on earlier pictures. Every segment starts with one.	Start of each segment

3. Architecture Diagrams

The system is shown in two diagrams. **Figure 1** shows the whole journey of a video, from the company's servers to the viewer's screen and back as measurements. **Figure 2** zooms inside the red “Adaptive Streaming Engine” box from Figure 1 and shows every part of the engine that was built.

How to read the diagrams

Each rounded box is one component. Each arrow shows data or a decision moving from one component to another.

The black numbered circles mark the order of the flow. The tables after each figure explain every box and every number.

Colours group related parts: blue = preparing/reading the video menu, green = network and delivery, orange = the viewer's device and buffer, purple = measurement, red = core decision making and error handling, grey = control.

Dashed arrows are feedback loops (information flowing back to improve a decision).

3.1 Figure 1: The end-to-end system

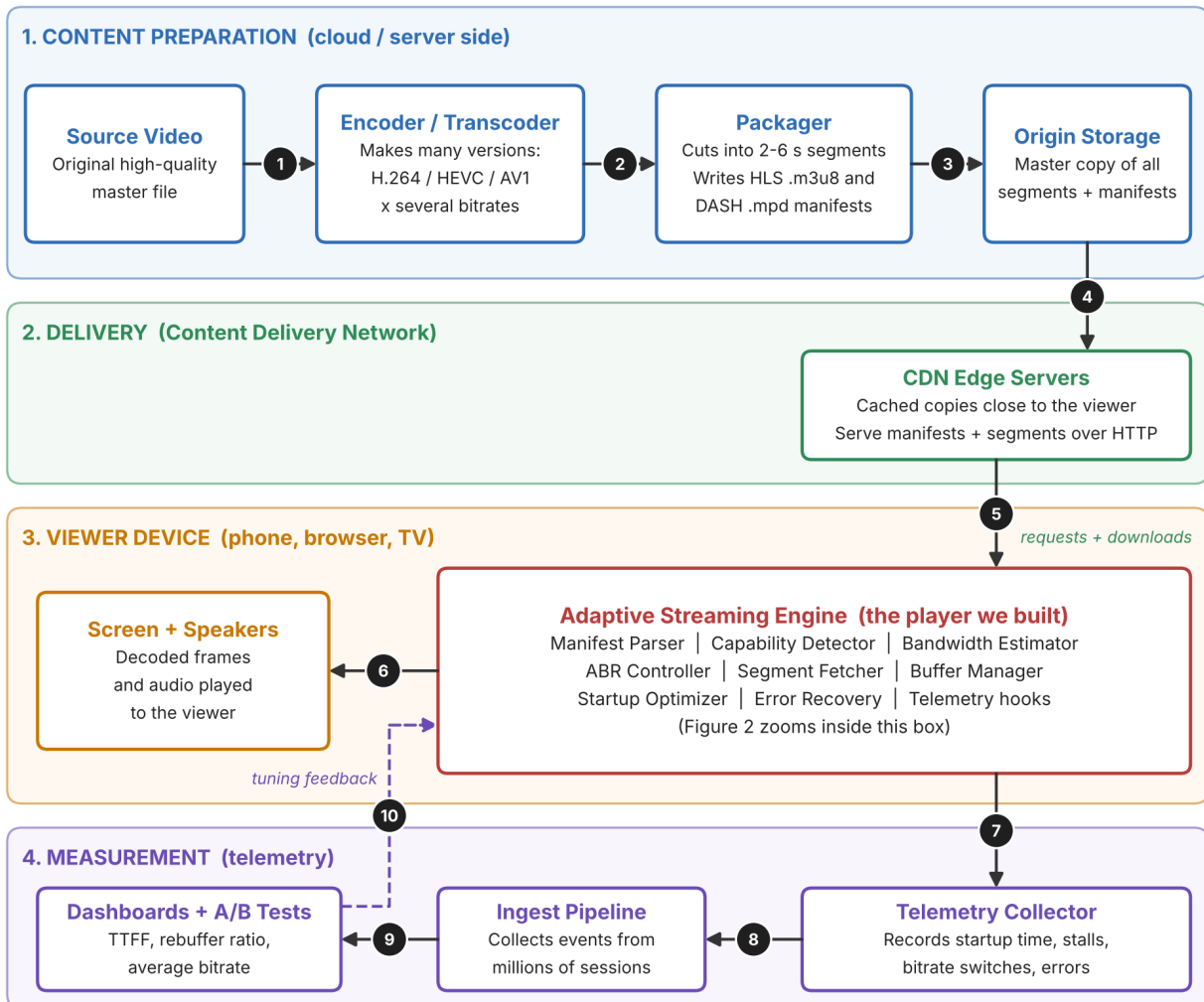


Figure 1. End-to-end architecture: content preparation, delivery, the viewer's device and measurement

The four layers (bands) of Figure 1

Layer	Where it runs	Its job in one sentence
1. Content preparation	Company's cloud servers	Turns one master video into many small pieces in many qualities and writes the menu files.
2. Delivery	CDN servers spread around the world	Keeps copies of the pieces close to viewers so they download fast.
3. Viewer device	Phone, browser, smart TV	Runs the adaptive streaming engine that picks, downloads and plays the right pieces.
4. Measurement	Device + company's analytics servers	Records how playback went for every viewer and turns it into charts and experiments.

Every box in Figure 1

Box	What it does, simply
Source Video	The original, very high-quality video file (the “master”) delivered by the content team.
Encoder / Transcoder	Compresses the master into many renditions: several resolutions and bitrates, each in H.264, HEVC and AV1.
Packager	Cuts every rendition into 2–6 second segments and writes the HLS (.m3u8) and DASH (.mpd) manifest files.
Origin Storage	The single source of truth that stores all segments and manifests.
CDN Edge Servers	Copies of the files on servers near viewers. Players download from here, not from the origin.
Adaptive Streaming Engine	The player logic built in this project. Decides what to download, downloads it, and feeds it to the screen.
Screen + Speakers	Where the decoded pictures and sound are finally shown to the viewer.
Telemetry Collector	Part of the player that records events: play pressed, first frame, stall, quality switch, error.
Ingest Pipeline	Servers that receive these events from millions of devices and store them.
Dashboards + A/B Tests	Charts of TTFF, rebuffering and bitrate; experiments that compare the old player with the new engine.

Every numbered arrow in Figure 1

#	From → To	What flows along the arrow
1	Source Video → Encoder	The master video file is sent for compression.
2	Encoder → Packager	Many compressed renditions (all qualities, all three codecs) with keyframes lined up at the same moments.
3	Packager → Origin	Segments plus manifest files are uploaded to storage.
4	Origin → CDN	CDN servers fetch and cache files the first time someone nearby asks for them.
5	CDN ↔ Engine	The engine requests the manifest and then segments over HTTP; the CDN sends the bytes back.
6	Engine → Screen	Decoded video frames and audio are played to the viewer.
7	Engine → Telemetry Collector	Every important playback event is recorded with a timestamp.
8	Collector → Ingest	Events are batched and sent to the company's servers.

#	From → To	What flows along the arrow
9	Ingest → Dashboards	Events are turned into metrics like TTFF and rebuffer ratio, sliced by device, network and region.
10	Dashboards → Engine (dashed)	Tuning feedback: what engineers learn from the data is used to adjust thresholds and settings in the engine.

3.2 Figure 2: Inside the adaptive streaming engine

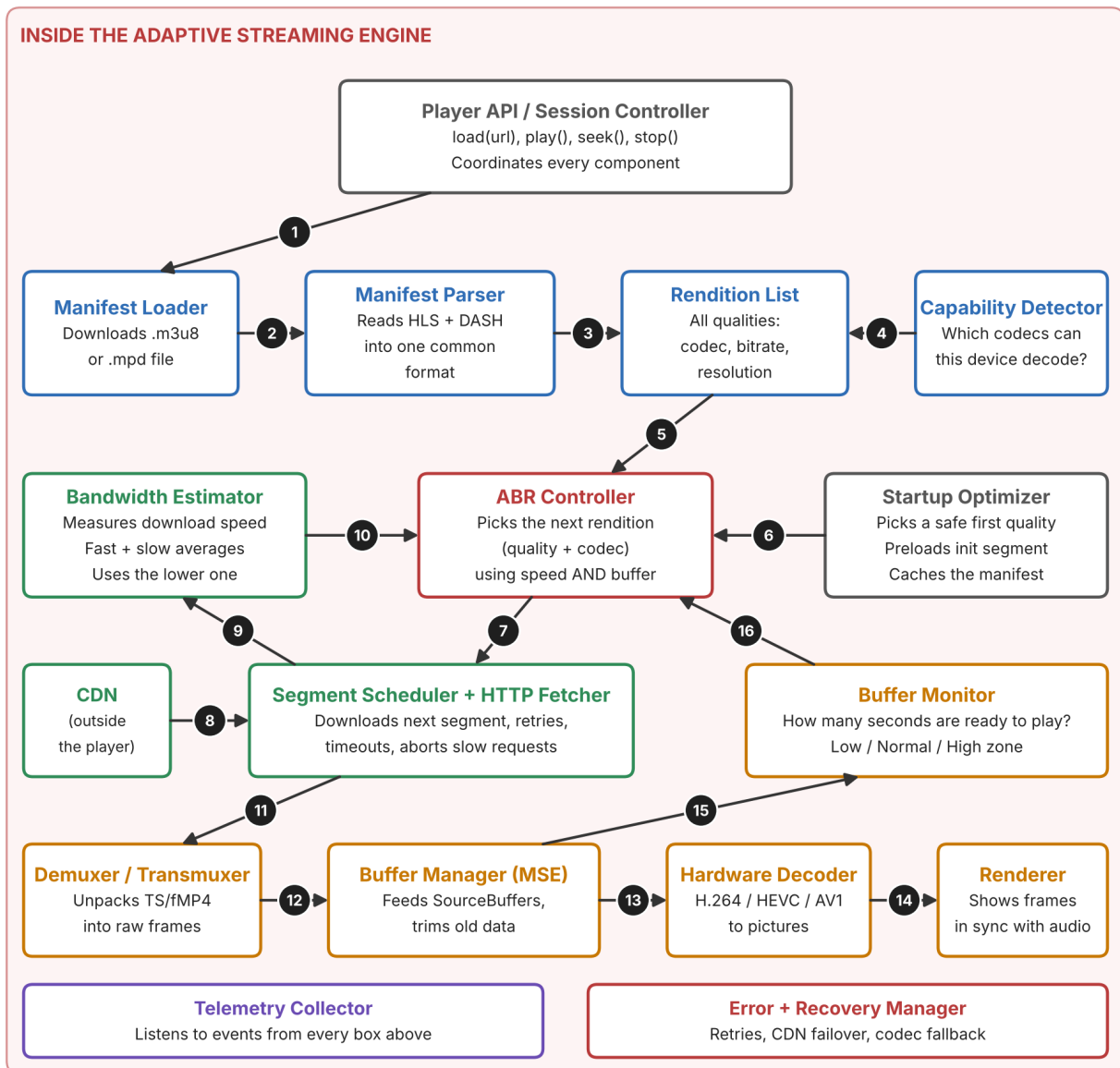


Figure 2. Internal architecture of the adaptive streaming engine

Every box in Figure 2

Box	What it does, simply
Player API / Session Controller	The front door. The app calls load, play, seek, stop. It starts the other parts and tracks the player's state.
Manifest Loader	Downloads the manifest file (.m3u8 or .mpd) from the CDN, with timeouts and retries.
Manifest Parser	Reads HLS or DASH text and converts both into one common internal format.
Rendition List	The common format: every available quality with its codec, bitrate, resolution and segment addresses.
Capability Detector	Asks the device which codecs and resolutions it can decode smoothly, and removes the rest from the list.
Bandwidth Estimator	Measures the speed of each download and keeps a fast and a slow average; uses the lower one to stay safe.
ABR Controller	The decision maker. Picks the next rendition (quality and codec) using both network speed and buffer level.
Startup Optimizer	Special rules for the first moments: picks a safe first quality, preloads the init segment, reuses cached data.
CDN	Not part of the player; shown to make clear where the bytes come from.
Segment Scheduler + HTTP Fetcher	Downloads the chosen segments in the right order, handles retries and aborts downloads that are too slow.
Buffer Monitor	Keeps checking how many seconds of video are saved up and reports a Low, Normal or High zone.
Demuxer / Transmuxer	Unwraps downloaded files into separate audio and video streams, converting old MPEG-TS files into fMP4 when needed.
Buffer Manager (MSE)	Feeds the bytes into the browser or device media pipeline and removes old, already-watched data.
Hardware Decoder	The device's chip that turns compressed H.264, HEVC or AV1 data into actual pictures.
Renderer	Shows each picture at the right moment, in sync with the sound.
Telemetry Collector	Listens to events from every box and records them for QoE measurement.
Error + Recovery Manager	Handles failures: retries, switches to a backup CDN, falls back to an older codec, unsticks a frozen player.

Every numbered arrow in Figure 2

#	From → To	What flows along the arrow
1	Player API → Manifest Loader	The app called load(url); the loader is told which manifest to fetch.
2	Loader → Parser	Raw manifest text.
3	Parser → Rendition List	A clean, format-independent list of tracks, renditions and segments.
4	Capability Detector → Rendition List	A filter: renditions the device cannot decode smoothly are removed.
5	Rendition List → ABR Controller	The final set of choices the ABR is allowed to pick from.
6	Startup Optimizer → ABR Controller	The starting choice for the very first segment and the startup buffer target.
7	ABR Controller → Fetcher	The decision: “download segment N of rendition R next”.
8	CDN → Fetcher	The bytes of the requested segment.
9	Fetcher → Bandwidth Estimator	Download measurements: bytes received and time taken for each segment.
10	Bandwidth Estimator → ABR Controller	The current safe bandwidth estimate (for example, 4.2 Mbps).
11	Fetcher → Demuxer	The downloaded segment file.
12	Demuxer → Buffer Manager	Separated audio and video data ready for the media pipeline.
13	Buffer Manager → Decoder	Compressed frames, in order, from the media buffer.
14	Decoder → Renderer	Decoded pictures ready to show.
15	Buffer Manager → Buffer Monitor	How far ahead of the playhead the saved video reaches.
16	Buffer Monitor → ABR Controller	The buffer zone (Low / Normal / High), which changes how bold or careful the next decision is.

Two loops make the engine adaptive. The network loop (arrows 7 → 8 → 9 → 10 → back to 7) tells the ABR how fast data is arriving. The buffer loop (arrows 11 → 12 → 15 → 16 → back to 7) tells the ABR how much safety margin is left. Every few seconds both loops feed a new decision.

4. Every Component in Detail

Each component below is described with the same five parts: what it is, why it is needed, how it works step by step, important details, and an everyday comparison. Components 4.1 to 4.3 prepare and deliver the video; 4.4 to 4.15 are inside the engine; 4.16 is the measurement system.

Part A: Preparing and delivering the video

4.1 Encoder / Transcoder and the bitrate ladder

What it is

A server program (for example FFmpeg-based or a cloud encoding service) that takes the master video and produces many compressed versions of it, called renditions. The full set of renditions is the bitrate ladder.

Why it is needed

A single version cannot suit everyone. A phone on weak 4G needs a small file; a 4K TV on fibre wants a large, sharp one. Producing several codecs lets each device use the most efficient format it supports.

How it works, step by step

- The master file is decoded into raw pictures.
- The pictures are resized to each target resolution (for example 240p, 360p, 480p, 720p, 1080p).
- Each resolution is compressed at a target bitrate with each codec: H.264, HEVC and AV1.
- Keyframes are forced at exactly the same timestamps in every rendition (for example every 2 seconds). This is called **GOP alignment**.
- Audio is encoded separately (for example AAC at 64 and 128 kbps) so it can be shared by all video renditions.

Important details

- **Why keyframe alignment matters:** the player can only switch quality at a keyframe. If all renditions put keyframes at the same moments, segment N of 480p and segment N of 1080p start at the same instant, so switching is seamless.
- **Codec trade-off:** AV1 gives the smallest files but needs newer hardware; H.264 is biggest but plays everywhere. Keeping all three means every device gets the best one it can handle.
- **Per-title tuning:** a cartoon needs fewer bits than a fast football match. The ladder can be tuned per video for efficiency.

Example bitrate ladder (illustrative)

Step	Resolution	H.264	HEVC	AV1
1	426 × 240	400 kbps	280 kbps	220 kbps
2	640 × 360	800 kbps	550 kbps	430 kbps
3	854 × 480	1,400 kbps	950 kbps	750 kbps

Step	Resolution	H.264	HEVC	AV1
4	1280 × 720	3,000 kbps	2,000 kbps	1,600 kbps
5	1920 × 1080	6,000 kbps	4,000 kbps	3,200 kbps

In everyday words

Think of printing the same book in pocket, standard and large-print editions, and in three languages. Every reader gets the edition that suits their eyes and the language they can read.

4.2 Packager and manifests (HLS and DASH)

What it is

A tool that cuts each rendition into small segments and writes the manifest files that describe them. It produces an HLS playlist (.m3u8) and a DASH manifest (.mpd) that point at the same segment files.

Why it is needed

Small segments make switching possible (every few seconds the player can choose again). Manifests are the menu the player reads to know what exists and where it is.

How it works, step by step

- Each rendition is cut at its keyframes into segments of equal length (for example 4 seconds).
- Segments are written in **CMAF / fragmented MP4** format so the same files work for both HLS and DASH.
- An **init segment** is written per rendition, containing codec settings the decoder needs first.
- The HLS **multivariant (master) playlist** lists every rendition with its bandwidth, resolution and codec string; each rendition has its own **media playlist** listing its segments.
- The DASH **MPD** describes the same thing in XML: Period → AdaptationSet (video or audio) → Representation (one rendition) → segment template.

Important details

- **Codec strings** tell the player exactly which codec profile is used, for example **avc1.640028** (H.264), **hvc1.1.6.L120.90** (HEVC) and **av01.0.08M.08** (AV1). The Capability Detector checks these strings against the device.
- **Live video:** for live streams the playlists are updated every few seconds with new segments, and the player reloads them.

What a tiny HLS master playlist looks like

```
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1280x720,CODECS="avc1.640028,mp4a.40.2"
video/h264_720p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=2000000,RESOLUTION=1280x720,CODECS="hvc1.1.6.L120.90,mp4a.40.2"
video/hevc_720p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1600000,RESOLUTION=1280x720,CODECS="av01.0.08M.08,mp4a.40.2"
video/av1_720p/playlist.m3u8
```

Each pair of lines says: “this rendition needs about this much bandwidth, has this size and codec, and its segment list is at this address.”

In everyday words

The packager is like slicing a long loaf of bread into equal slices and writing a menu that says which slice is which, in two languages (HLS and DASH), so any customer can order.

4.3 Origin storage and CDN

What it is

The origin is the central storage holding every segment and manifest. The CDN is a network of edge servers worldwide that keep cached copies close to viewers.

Why it is needed

Downloading from a server on another continent is slow. A nearby edge server makes every request faster, which directly reduces startup time and stalls.

How it works, step by step

- A player asks the CDN for a file using a normal HTTP request.
- If the nearby edge already has the file (a **cache hit**), it sends it immediately.
- If not (a **cache miss**), the edge fetches it from the origin once, stores it, and sends it on. Later viewers get the cached copy.

Important details

- Segments never change, so they are cached for a long time. Live playlists change constantly, so they are cached for only a second or two.
- Using more than one CDN allows the engine to fail over if one CDN is slow or broken (see 4.15).
- Low **time to first byte** from the CDN is a big part of good TTFF.

In everyday words

The origin is the central warehouse; CDN edges are local shops. You buy from the shop around the corner, which restocks from the warehouse only when it runs out.

Part B: Inside the adaptive streaming engine

4.4 Player API and Session Controller

What it is

The public interface the app uses (`load()`, `play()`, `pause()`, `seek()`, `stop()`) and the internal coordinator that starts every other component and keeps track of the player's state.

Why it is needed

Many components must act in the right order. A single coordinator avoids confusion, such as downloading segments before the manifest is read.

How it works, step by step

- The app calls `load(ur1)`. The controller records the time (used later for TTFF) and starts the Manifest Loader.
- When the rendition list is ready, it asks the ABR for the first choice and tells the Fetcher to start.
- It moves the player through states and announces every change to the Telemetry Collector.
- On `seek()` it cancels in-flight downloads, clears the buffer as needed and restarts fetching at the new time.
- On `stop()` it releases the network connections, decoder and memory.

Player states (state machine)

State	Meaning	Moves to
IDLE	Nothing loaded yet	LOADING
LOADING	Fetching and reading the manifest	BUFFERING or ERROR
BUFFERING	Waiting for enough video to start (startup)	PLAYING
PLAYING	Video is moving on screen	REBUFFERING, PAUSED, SEEKING, ENDED
REBUFFERING	Buffer ran empty; frozen while waiting (a stall)	PLAYING or ERROR
SEEKING	Jumping to a new position	BUFFERING
PAUSED / ENDED / ERROR	User paused, video finished, or unrecoverable failure	PLAYING / IDLE / IDLE

In everyday words

The session controller is the conductor of an orchestra. It does not play an instrument; it tells every player when to start and stop.

4.5 Manifest Loader

What it is

The part that downloads the manifest file (.m3u8 or .mpd) from the CDN.

Why it is needed

Nothing can happen until the menu is known, so this download sits directly on the startup path and must be fast and reliable.

How it works, step by step

- Sends an HTTP GET for the manifest URL with a short timeout.
- If it fails, retries with a short, growing delay (for example 250 ms, 500 ms, 1 s), and can switch to a backup CDN address.
- For HLS it fetches the master playlist and then only the media playlist(s) it actually needs, not all of them.
- For live streams it reloads the playlist on a timer (about once per segment length) to learn about new segments.

Important details

- A recently loaded manifest can be kept in memory, so replaying or preloading (for example when a video thumbnail is on screen) skips this network trip. This is one of the startup speed-ups.

In everyday words

Like picking up the restaurant menu before ordering. Keeping last time's menu in your pocket saves a trip to the counter.

4.6 Manifest Parser and the unified rendition model

What it is

Code that reads HLS text or DASH XML and converts both into one common internal structure: tracks (video, audio, subtitles) → renditions (codec, bitrate, resolution, frame rate) → segments (URL, start time, duration).

Why it is needed

HLS and DASH describe the same idea with different syntax. Converting both into one model means the ABR, Fetcher and Buffer logic are written only once and work for both formats.

How it works, step by step

- Detects the format from the file content or extension.
- HLS: reads `#EXT-X-STREAM-INF` lines for renditions and `#EXTINF` lines for segment durations.
- DASH: reads the XML tree and expands segment templates (for example `seg_${Number$.m4s}`) into a real segment list.
- Fills in the common model and validates it (missing codecs, zero durations, broken URLs).
- Hands the model to the Rendition List, where the Capability Detector filters it.

In everyday words

A translator who turns a French menu and an English menu into one standard order form that the kitchen understands.

4.7 Capability Detector (codec support)

What it is

A component that asks the device what it can decode well: which codecs, which profiles, what resolution and frame rate, and whether decoding is hardware-accelerated.

Why it is needed

Choosing a rendition the device cannot decode causes errors or a black screen. Choosing one it can only decode in software drains battery and drops frames. And sending 1080p to a small 360-pixel-wide player wastes bandwidth.

How it works, step by step

- For each rendition's codec string, asks the platform whether it is supported (in browsers: `MediaSource.isTypeSupported()` and `MediaCapabilities.decodingInfo()`, which also answers “smooth?” and “power-efficient?”).
- Removes renditions that are unsupported, not smooth, or larger than the screen/player size needs.
- Ranks codec families: prefers AV1, then HEVC, then H.264, but only if the device decodes them smoothly and power-efficiently.
- Saves the result so the next session on the same device skips the checks.

Important details

- Same picture quality at fewer bits means a more efficient codec can deliver a higher quality step on the same connection, or the same quality with more safety margin, which reduces stalls.

In everyday words

Before handing someone a disc, you check whether their machine plays Blu-ray, DVD or only CD, and you hand them the best format their machine plays smoothly.

4.8 Bandwidth Estimator

What it is

The component that measures how fast data is arriving and predicts how fast the next download will be.

Why it is needed

To choose a quality, the ABR must know whether the connection can deliver that quality's bitrate in time. A bad estimate either causes stalls (too optimistic) or blurry video (too pessimistic).

How it works, step by step

- For every downloaded segment it records bytes received and time taken. One sample = $\text{bytes} \times 8 \div \text{seconds}$ (bits per second).
- Ignores samples that are too small to be meaningful (for example under 16 KB) or that came instantly from a cache.

- Feeds each sample into **two moving averages**: a **fast** one that reacts within a couple of segments and a **slow** one that changes gently.
- Uses the **lower** of the two. A sudden drop is caught quickly by the fast average; a sudden spike is not trusted until the slow average agrees.
- Multiplies by a **safety factor** (for example 0.85) to leave headroom.
- Before any download has happened, starts from the last session's saved estimate (or a conservative default) instead of guessing high.

Worked example (fast weight 0.5, slow weight 0.15, safety factor 0.85)

Segment	Measured speed	Fast average	Slow average	Lower one	Safe estimate
Start	—	8.00 Mbps	8.00 Mbps	8.00	6.80 Mbps
1	2 Mbps (sudden drop)	5.00	7.10	5.00	4.25 Mbps
2	2 Mbps	3.50	6.34	3.50	2.98 Mbps
3	8 Mbps (recovering)	5.75	6.59	5.75	4.89 Mbps
4	8 Mbps	6.88	6.80	6.80	5.78 Mbps

Notice how the estimate drops immediately when the network gets worse (rows 1–2) but rises carefully when it gets better (rows 3–4). That asymmetry is what prevents stalls.

In everyday words

Like judging traffic before a trip: you react at once to news of a jam, but you do not believe the road is clear until it has stayed clear for a while.

4.9 Buffer Monitor

What it is

A component that continuously measures how many seconds of video are downloaded ahead of what is currently playing.

Why it is needed

The buffer is the safety cushion. Even if the network briefly stops, playback continues while the buffer lasts. Its level tells the ABR how much risk it can take.

How it works, step by step

- Reads the buffered time ranges from the media pipeline and subtracts the current play position.
- Places the result in a zone and tells the ABR whenever the zone changes.
- Tells the Fetcher to pause downloading when the buffer is full, to save data and memory.

Buffer zones (illustrative values)

Zone	Seconds ahead	What the engine does
Danger	Under 5 s	Drop quality right away, possibly several steps; abandon slow downloads.
Normal	5 – 20 s	Follow the bandwidth estimate; small, careful changes only.
Comfortable	Over 20 s	Allowed to try one step higher quality.
Full	30 s	Stop downloading until some video has been played.

In everyday words

Like the fuel gauge in a car. When it is low you drive gently to the next station; when the tank is full you can enjoy the drive.

4.10 ABR Controller (rendition switching)**What it is**

The decision maker. Before every segment it chooses which rendition to download, meaning which quality level and which codec.

Why it is needed

This is where the user experience is won or lost. Good choices give the sharpest picture the connection can sustain without ever running the buffer dry.

How it works, step by step

- Starts with the allowed renditions (after the Capability Detector has filtered them) for the chosen codec family.
- Takes the safe bandwidth estimate and finds the highest bitrate that fits under it.
- Adjusts using the buffer zone: in Danger it drops lower than bandwidth alone suggests; in Comfortable it may go up one step.
- Applies **hysteresis**: going down happens immediately, going up only if the higher step would still fit with margin and at least about 10 seconds have passed since the last up-switch. This stops flickering between qualities.
- Applies **download abandonment**: if a segment is taking so long that the buffer would run out before it finishes, the download is cancelled and a lower rendition is fetched instead.
- Sends the decision to the Segment Fetcher and reports the switch to telemetry.

Important details

- **Why both bandwidth and buffer**: bandwidth alone overreacts to every small dip; buffer alone reacts too late. Combining them (a hybrid approach) is steadier and safer.

- **Codec switching:** normally the engine stays within one codec family for a session. It switches family (for example AV1 to HEVC to H.264) when decoding fails or performs badly. In browsers this uses `SourceBuffer.changeType()` so the switch happens without restarting playback.
- **Seamless switches:** because keyframes are aligned (4.1), segment N of any rendition starts at the same time, so the new quality simply continues where the old one stopped.

Example decisions

Safe bandwidth	Buffer	Current	Decision	Why
4.9 Mbps	22 s (Comfortable)	720p	Stay 720p	1080p H.264 needs 6 Mbps; does not fit.
4.9 Mbps	22 s (Comfortable)	720p AV1	Up to 1080p AV1	1080p AV1 needs 3.2 Mbps; fits with margin.
2.9 Mbps	12 s (Normal)	720p H.264	Down to 480p	720p needs 3 Mbps; just above the estimate.
2.9 Mbps	3 s (Danger)	720p H.264	Down to 360p	Danger zone: drop extra to refill the buffer fast.

In everyday words

The ABR is the driver choosing the gear. It looks at the road (bandwidth) and the fuel gauge (buffer) together, shifts down at once on a hill, and shifts up only when it is sure.

4.11 Segment Scheduler and HTTP Fetcher

What it is

The component that actually downloads segments over HTTP, in the right order and at the right time.

Why it is needed

Downloads must arrive before they are needed, without wasting data, and survive network hiccups.

How it works, step by step

- Receives the ABR decision: rendition R, segment N.
- Builds the URL (or byte range) from the rendition model and sends the request, reusing open connections (HTTP keep-alive / HTTP/2) to avoid repeated connection setup.
- Downloads audio and video segments in parallel when they are separate tracks.
- Streams progress to the Bandwidth Estimator and the ABR (needed for abandonment decisions).
- On failure retries with backoff; after repeated failure asks the Error Manager to switch CDN.
- Hands the finished segment to the Demuxer, then immediately asks the ABR for the next decision, unless the buffer is Full.

In everyday words

A delivery driver who picks up exactly the next box needed, takes the fastest road, and if a road is blocked, takes a detour or calls the depot.

4.12 Startup Optimizer (the TTFF improvement)**What it is**

A set of rules that only apply between pressing Play and the first frame. It is the main reason time-to-first-frame dropped by 38%.

Why it is needed

At startup the engine knows nothing about the network yet, and the viewer is waiting and staring at a black screen. Every round trip and every extra kilobyte here is felt directly.

How it works, step by step

- **Safe first rendition:** starts at a modest quality (for example 480p or the quality the saved bandwidth estimate supports) instead of the top, so the first segment downloads quickly. The ABR climbs within seconds once real measurements arrive.
- **Remembered bandwidth:** reuses the estimate saved from the previous session on this device.
- **Parallel startup fetches:** downloads the init segment and the first media segment at the same time instead of one after another.
- **Low start threshold:** begins playback once a small amount (for example about 2 seconds) is buffered rather than waiting for a large buffer.
- **Warm-up before Play:** opens the CDN connection early and preloads/caches the manifest when a video is likely to be played (for example its thumbnail is visible).
- **Cached capability results:** codec checks from the last session are reused.

In everyday words

Like starting a car in first gear rather than fifth. You get moving immediately and shift up once you are rolling.

4.13 Demuxer / Transmuxer**What it is**

Code that unpacks downloaded segment files into separate audio and video data with timestamps, and converts older formats into the one the media pipeline accepts.

Why it is needed

Browsers' media pipeline (MSE) accepts fragmented MP4. Older HLS streams use MPEG-TS files, which must be re-wrapped (transmuxed) first. Newer CMAF streams need little or no conversion.

How it works, step by step

- Reads the container headers and finds audio and video samples.
- Extracts timestamps and marks keyframes.
- If the input is MPEG-TS, re-wraps the same compressed samples into fMP4 boxes (no re-encoding, so it is fast).
- Passes the result to the Buffer Manager.

In everyday words

Like unpacking a mixed parcel and sorting its contents into the right drawers, with labels saying when each item will be needed.

4.14 Buffer Manager (Media Source Extensions)

What it is

The component that feeds bytes into the device's media pipeline (in browsers, MSE `SourceBuffer` objects, one for video and one for audio) and keeps that buffer healthy.

Why it is needed

The decoder can only play what has been appended to these buffers. They also have limited memory, so old data must be cleaned up.

How it works, step by step

- On first use of a rendition, appends its init segment, then appends media segments in order.
- Waits for each append to finish (`updateend`) before the next one, keeping a small queue.
- Removes already-watched data more than about 30 seconds behind the playhead to free memory.
- If the device reports the buffer is full (`QuotaExceededError`), removes more old data and retries.
- Detects small gaps between segments and jumps the playhead over them so playback does not stall on a gap.
- For a codec switch, calls `changeType()` with the new codec string, then appends the new init segment.

In everyday words

Like a conveyor belt feeding a machine: it keeps the belt loaded just ahead of the machine and clears away the empty trays behind it.

4.15 Decoder, Renderer, and the Error + Recovery Manager

What it is

The hardware decoder turns compressed frames into pictures; the renderer shows them at the right time, synchronised to the audio clock. The Error + Recovery Manager watches for anything going wrong and fixes it automatically where possible.

Why it is needed

Decoding must keep up in real time. And on millions of devices, failures are certain; recovering quietly prevents stalls and abandoned sessions.

How it works, step by step

- The decoder decodes each frame (using the device's chip for H.264, HEVC or AV1) and the renderer presents it when the audio clock reaches its timestamp.
- Dropped frames are counted; many drops mean the rendition is too heavy for the device, and the ABR caps quality.
- The Error Manager classifies every failure and applies the matching fix (see table).
- A **stall watchdog** checks if the playhead stops moving even though data is buffered, and gives it a small nudge forward.

Error handling table

Problem	Automatic recovery
Network timeout or 5xx server error	Retry with growing delay; after several failures switch to the backup CDN.
Segment missing (404)	Try the same segment from another rendition or CDN; skip forward only as a last resort.
Decode error / unsupported codec	Fall back to the next codec family: AV1 → HEVC → H.264.
Too many dropped frames	Cap the maximum quality for this device for the rest of the session.
Player stuck with data buffered	Watchdog nudges the playhead past the gap.
Live playlist not updating	Reload from backup CDN; move to the newest available segment.

In everyday words

The decoder and renderer are the projector. The Error Manager is the projectionist who fixes a jammed reel before the audience notices.

Part C: Measuring the experience

4.16 QoE Telemetry: collector, pipeline and dashboards

What it is

The system that records what happened during every playback session and turns it into trustworthy numbers: TTFF, rebuffering and more. The 38% and 42% results come from here.

Why it is needed

Without measurements from real viewers, nobody could know whether the new engine is actually better, on which devices, or which setting to tune next.

How it works, step by step

- The Telemetry Collector inside the player listens to events: `play_requested`, `first_frame`, `stall_start`, `stall_end`, `bitrate_switch`, `error`, `session_end`, each with a precise timestamp and context (device, network type, codec, CDN).
- Events are batched and sent in the background (and flushed when the app closes) so measuring does not slow playback.
- The ingest pipeline collects events from millions of sessions, cleans them (removes duplicates, rejects impossible values) and stores them.
- Metrics are computed per session and then aggregated (median, 90th percentile) by device, region, network and player version.
- Dashboards show the trends; A/B tests send part of the traffic to the new engine and compare it with the old one under the same conditions.

How the main metrics are calculated

Metric	Formula in words	Notes
Time-to-first-frame	time of <code>first_frame</code> – time of <code>play_requested</code>	Reported as median and 90th percentile.
Rebuffer ratio	total stall time ÷ (watch time + stall time)	Startup wait and seek waits are excluded; only unexpected freezes count.
Rebuffer frequency	number of stalls per hour of viewing	Shows how often, not just how long.
Average bitrate	time-weighted average of the bitrate played	Proves quality was not sacrificed.
Switch count	number of quality changes per session	Too many switches feel unstable.
Exits before start	sessions closed before the first frame	Falls when TTFF improves.

In everyday words

Like a fitness tracker for every video session. It records each step, and the dashboards are the weekly report showing whether the new training plan works.

5. The Complete Workflow, End to End

This section follows one video from the moment it is uploaded to the moment its playback statistics appear on a dashboard. The workflow is split into phases; each phase is a table of steps in order, with a plain-language explanation for each step.

Phase A: Preparing the video (before anyone presses Play)

#	What happens	In simple words
1	The master video file is uploaded to the encoding system.	The original, best-quality copy arrives.
2	The encoder creates every rendition: each resolution in H.264, HEVC and AV1, with keyframes aligned every 2 seconds.	Many sizes and formats of the same video are made, all lined up in time.
3	Audio is encoded once at a few bitrates.	Sound is prepared separately so all video versions can share it.
4	The packager cuts everything into segments (CMAF/fMP4) and writes init segments.	The video is sliced into small, equal pieces.
5	The packager writes the HLS .m3u8 playlists and the DASH .mpd manifest.	Menus are written in both formats.
6	Everything is uploaded to origin storage, ready for CDN edges to fetch on demand.	The pieces are placed in the warehouse, ready for local shops.

Phase B: The viewer presses Play (startup)

#	What happens	In simple words
1	The app calls <code>load(url)</code> and <code>play()</code> . The controller records the time as <code>play_requested</code> .	The stopwatch for startup time begins.
2	The Startup Optimizer loads the saved bandwidth estimate and cached codec results from the last session.	The player remembers how good this device's network and decoder were last time.
3	The Manifest Loader fetches the manifest (or uses a preloaded copy).	The menu is picked up.
4	The Parser converts HLS or DASH into the common rendition model.	The menu is translated into the engine's own language.
5	The Capability Detector removes renditions the device cannot decode smoothly and picks the codec family.	Only playable options remain, with the most efficient codec preferred.

#	What happens	In simple words
6	The Startup Optimizer and ABR pick a safe first rendition.	Start in a low gear so the first piece arrives fast.
7	The Fetcher downloads the init segment and first media segment in parallel.	Two downloads at once instead of one after the other.
8	The Bandwidth Estimator records the first real speed measurement.	Now the engine has fresh information about the network.
9	The Demuxer unpacks the segment; the Buffer Manager appends init and media data to the SourceBuffers.	The pieces are put on the conveyor belt to the decoder.
10	When about 2 seconds are buffered, playback starts.	Do not wait for a big pile, start as soon as it is safe.
11	The decoder outputs the first picture; the renderer shows it. The controller records first_frame .	The first frame appears and the stopwatch stops. TTFF = this time minus step 1.

Startup timeline example (illustrative)

Time after Play	Old player	New engine
0 ms	Play pressed	Play pressed (manifest already preloaded)
0 – 300 ms	Fetch master playlist	Read cached manifest; pick safe rendition
300 – 600 ms	Fetch media playlist	Init + first segment downloading in parallel
600 – 900 ms	Fetch init segment	Segments arrive; append to buffer
about 1.3 s	Fetch first segment at high quality (slow)	First frame shown
about 2.1 s	First frame shown	Already playing; ABR climbing to higher quality

Phase C: Normal playback (the loop that repeats every segment)

#	What happens	In simple words
1	The Buffer Monitor reports the current buffer level and zone.	Check the fuel gauge.
2	The Bandwidth Estimator provides the current safe estimate.	Check the traffic report.
3	The ABR combines both, applies hysteresis, and picks rendition R for segment N.	Choose the gear for the next stretch of road.

#	What happens	In simple words
4	The Fetcher downloads segment N of rendition R, reporting progress as it goes.	Fetch the next piece.
5	If the download is too slow to finish before the buffer empties, it is abandoned and a lower rendition is fetched.	If the parcel is late, grab a smaller one that arrives in time.
6	The finished segment is measured (new bandwidth sample), demuxed and appended to the buffer.	Measure, unpack, put on the belt.
7	The Buffer Manager removes old watched data behind the playhead.	Clear away the empty trays.
8	Every switch and event is sent to the Telemetry Collector. If the buffer is Full, downloading pauses; otherwise the loop repeats.	Log what happened and go again.

Phase D: The network gets worse

#	What happens	In simple words
1	A segment download takes longer than usual; the fast bandwidth average drops at once.	The engine notices the slowdown within one piece.
2	Because the engine uses the lower of the two averages, the safe estimate falls immediately.	It believes bad news straight away.
3	The ABR switches down to a rendition that fits the new estimate, immediately (no waiting period for down-switches).	Shift down a gear right away.
4	If the buffer has fallen into the Danger zone, the ABR drops an extra step to refill faster.	When fuel is low, drive extra gently.
5	Because the new segment starts at an aligned keyframe, the picture becomes slightly softer but never freezes.	The viewer sees a bit less sharpness instead of a spinning wheel.

Phase E: The network gets better

#	What happens	In simple words
1	Downloads get faster; the fast average rises but the slow average lags behind.	Good news is noted but not yet fully trusted.
2	The safe estimate rises gradually as the slow average catches up.	Wait until the improvement is real.

#	What happens	In simple words
3	When the buffer is Comfortable, the next step up fits with margin, and enough time has passed since the last up-switch, the ABR moves up one step.	Shift up one gear at a time, only when sure.
4	The process repeats until the best sustainable quality is reached.	Keep climbing until the connection's limit.

Phase F: Codec fallback and switching

#	What happens	In simple words
1	At startup the engine prefers AV1, then HEVC, then H.264, depending on smooth hardware support.	Use the most efficient format the device handles well.
2	If decoding errors or many dropped frames occur, the Error Manager marks that codec family as bad for this session.	The device is struggling with this format.
3	The ABR selects the matching quality in the next codec family.	Same quality step, older but easier format.
4	The Buffer Manager calls <code>changeType()</code> with the new codec and appends the new init segment; playback continues at the next segment boundary.	Swap formats without stopping the video.

Phase G: The viewer seeks (jumps to another time)

#	What happens	In simple words
1	The app calls <code>seek(t)</code> . In-flight downloads are cancelled.	Stop fetching pieces that are no longer needed.
2	The engine finds the segment that contains time <code>t</code> in the rendition model.	Look up which piece holds the new moment.
3	The ABR chooses a rendition using the current estimate (buffer is empty at the new position, so it is cautious).	Start the new position in a safe gear.
4	The segment is fetched, appended, and playback resumes from the nearest keyframe. The wait is logged as seek time, not rebuffering.	The jump completes; telemetry counts it fairly.

Phase H: Errors happen

#	What happens	In simple words
1	A request fails or times out.	A piece did not arrive.
2	The Fetcher retries with a short, growing delay.	Try again, a little later each time.
3	After repeated failures the Error Manager switches to the backup CDN.	Use a different shop.
4	If the problem is decoding, it falls back to another codec (Phase F).	Use a format the device handles better.
5	Only if nothing works is a clear error shown to the viewer, and the error is reported to telemetry.	Give up gracefully and tell the team.

Phase I: Session ends, and the numbers reach the dashboard

#	What happens	In simple words
1	The viewer stops or the video ends. The collector sends <code>session_end</code> with totals (watch time, stall time, switches).	The trip summary is written.
2	Batched events reach the ingest pipeline and are cleaned and stored.	All trip reports are gathered centrally.
3	Per-session metrics are computed: TTFF, rebuffer ratio, average bitrate, switch count.	Each session gets its scores.
4	Metrics are aggregated and the A/B test compares the new engine with the old player.	Old and new players are compared fairly.
5	Results (TTFF –38%, rebuffering –42%) appear on dashboards; insights are used to tune thresholds, which flows back to the engine.	The proof is visible, and lessons improve the next version.

6. How the Improvements Were Achieved and Measured

6.1 What reduced time-to-first-frame by 38%

Technique	Component	Why it makes startup faster
Safe first rendition instead of top quality	Startup Optimizer + ABR	A smaller first segment downloads much faster.
Remembered bandwidth from last session	Bandwidth Estimator	A realistic first guess; no slow probing.
Init + first segment fetched in parallel	Segment Fetcher	Removes one full network round trip.
Manifest preloading and caching	Manifest Loader	Removes the manifest download from the critical path.
Early CDN connection warm-up	Fetcher	No connection-setup delay when Play is pressed.
Small start threshold (about 2 s)	Buffer Manager	Playback begins as soon as it is safe, not after a big buffer fills.
Cached codec capability results	Capability Detector	Skips device checks on repeat sessions.
Nearby CDN edge with good caching	CDN	Low time to first byte for every request.

6.2 What reduced rebuffering by 42%

Technique	Component	Why it prevents freezes
Lower of fast and slow averages, with safety factor	Bandwidth Estimator	Reacts instantly to drops; never over-trusts spikes.
Hybrid decisions using bandwidth and buffer	ABR Controller	Drops quality before the buffer runs dry.
Immediate down-switch, careful up-switch	ABR Controller	Avoids climbing into a quality the network cannot hold.
Download abandonment	ABR + Fetcher	A late segment is replaced by a smaller one before the buffer empties.
Efficient codecs (AV1/HEVC) where supported	Capability Detector	Same quality with fewer bits leaves more safety margin.
Retries and CDN failover	Error Manager	Server problems do not turn into freezes.
Gap jumping and stall watchdog	Buffer Manager + Error Manager	Stops the player getting stuck on small gaps.

6.3 How the numbers were measured

- **A/B test:** viewers were randomly split between the old player and the new engine, so both groups had the same mix of devices, networks and content.
- **Same definitions for both groups:** TTFF from `play_requested` to `first_frame`; rebuffering counts only unexpected stalls (not startup or seeks).
- **Percent improvement** = $(\text{old value} - \text{new value}) \div \text{old value}$. Example: TTFF $(2.1 - 1.3) \div 2.1 \approx 38\%$; rebuffer ratio $(1.2\% - 0.7\%) \div 1.2\% \approx 42\%$.
- **Quality guardrail:** average bitrate was tracked at the same time to show the gains did not come from simply lowering picture quality.

7. Explaining the Project in an Interview

7.1 The 30-second version

Say it like this

“I built the adaptive streaming engine for our video player. It plays both HLS and DASH by converting them into one internal model, checks which codecs the device can decode, and picks between H.264, HEVC and AV1 renditions. It estimates bandwidth with a fast and a slow moving average and combines that with the buffer level to choose the next segment's quality. A dedicated startup path, with a safe first rendition, parallel fetches and preloading, cut time-to-first-frame by 38%, and the more conservative, buffer-aware switching cut rebuffering by 42%, both measured through QoE telemetry in an A/B test.”

7.2 Likely follow-up questions with short answers

Question	Short answer
Why support both HLS and DASH?	Different devices and content partners use different formats. A unified internal model means one ABR and one pipeline serve both.
Why not always play the highest quality?	If the network cannot deliver it in time, the buffer empties and the video freezes. Viewers hate freezes more than slightly lower sharpness.
Why use the lower of two averages?	The fast one catches drops quickly; the slow one stops brief spikes from causing over-ambitious up-switches.
How did you avoid flickering between qualities?	Hysteresis: down-switch immediately, up-switch one step at a time only with margin and a minimum time since the last up-switch.
How does switching stay seamless?	All renditions have keyframes at the same timestamps, so segment N of any rendition starts at the same moment.
How do you choose the codec?	Capability checks for smooth, power-efficient hardware decoding; prefer AV1, then HEVC, then H.264; fall back on errors using <code>changeType()</code> .
How do you know the gains were real?	A/B test on real traffic, same metric definitions for both groups, medians and 90th percentiles, with average bitrate as a guardrail.
What would you improve next?	Per-device and per-network tuning from telemetry, smarter prefetching, and low-latency live streaming support.